

Project	Kipepeo Wallet — Hot Wallet Validator
Date	2026-04-29 17:48 UTC
Files	hot_wallet.ak, identity_escrow.ak
Plan	Pro — 179 ADA
Score	87 / 100
Mainnet	Corrections recommended before mainnet
Standard	CIP-0052 Cardano Audit Guidelines
Audited by	AikenGuard v0.5 — 21 detectors + Qwen 3.5 27B + RAG Cardano

Summary

Severity	Count	Description
CRITICAL	0	No critical vulnerabilities found
HIGH	0	No high severity issues
MEDIUM	2	Logic issues requiring attention
LOW	5	Quality and documentation improvements
TOTAL	7	Findings across all validators

Layer 1 — Static Analysis (21 rules)

AK-017	[LOW]	Validator without documentation	hot_wallet.ak:12
This validator does not contain any documentation comments. CIP-0052 recommends that each validator documents its role, parameters, and validation conditions. Code: <pre>validator { fn spend(datum: Datum, redeemer: Redeemer, ctx: ScriptContext) -> Bool {</pre>			
Recommendation: Add /// comments before the validator describing expected invariants, parameters, and return conditions.			
AK-019	[LOW]	Variable with too short name in validator	hot_wallet.ak:18

A variable named with a single character (a, b, x, y) makes the code difficult to read and audit. A human auditor cannot understand the intent of the code without explicit context.

Code:

```
let tx = ctx.transaction
let a = tx.extra_signatories
```

Recommendation: Use descriptive names: instead of 'let x = ...', write 'let input_value = ...'

AK-020	[MEDIUM]	Validator logic too complex	identity_escrow.ak:24
--------	----------	-----------------------------	-----------------------

This validator contains overly complex logic in a single block. Complex validators are difficult to audit and maintain. Complexity increases the risk of hidden bugs.

Code:

```
fn escrow(datum: EscrowDatum, redeemer: EscrowRedeemer, ctx: ScriptContext) ->
  Bool {
    // 45 lines of mixed authorization, value, datum and time logic
    ...
```

Recommendation: Decompose into auxiliary functions: fn check_authorization(...), fn check_value(...), fn check_datum(...) called from the main validator.

AK-012	[MEDIUM]	Datum not persisted in continuing output	identity_escrow.ak:67
--------	----------	--	-----------------------

The validator modifies state but does not verify that the datum is correctly carried forward in the continuing output. This can allow an attacker to reset or corrupt the contract state.

Code:

```
let outputs = ctx.transaction.outputs
expect [output] = list.filter(outputs, ...)
// Missing: datum continuity check
```

Recommendation: Add explicit datum continuity check: verify that the output datum matches the expected updated datum.

Layer 2 — AI Deep Analysis + RAG Cardano

Analysis performed by Qwen 3.5 27B with RAG context from Vacuumlabs CTF (26 challenges), Vacuumlabs audit reports, Aiken stdlib, and Aikido Security (75 detectors).

Overall Assessment:

The hot_wallet and identity_escrow validators show a reasonable security baseline. The main concern is the interaction between the two contracts: the identity_escrow validator does not verify that the identity datum used for authorization matches the hot_wallet datum owner. This creates a potential privilege escalation vector where an attacker could use a valid identity from another user to authorize a spend from a different hot wallet.

Severity	Title	Affected Contracts	Reference
MEDIUM	Cross-contract identity mismatch	hot_wallet.ak + identity_escrow.ak	Vacuumlabs audit — AADA Finance

Security Score	87 / 100	Mainnet Status	Corrections recommended
----------------	----------	----------------	-------------------------

